

HPhi MPI 通信バッチ処理最適化

技術レポート

HPhi Development Team

February 27, 2026

Abstract

本レポートでは、HPhi 量子格子モデルシミュレータにおける MPI 通信の最適化手法について説明する。複数の転送項や InterAll 相互作用項が同じ MPI 通信相手 (origin) を持つ場合、これらをグループ化して MPI 通信回数を削減するバッチ処理を実装した。本手法により、特に大規模系・多プロセス環境での MPI 通信オーバーヘッドを大幅に削減できる。

Contents

1	はじめに	3
2	問題の背景	3
2.1	HPhi における MPI 並列化	3
2.2	転送項の MPI 通信	3
2.3	従来実装の問題点	3
3	最適化手法：バッチ処理	3
3.1	基本アイデア	3
3.2	アルゴリズム	4
4	データ構造	4
4.1	転送項グループ (MPITransferGroup)	4
4.2	バッチコンテナ (MPIBatchedTransfers)	4
4.3	MPIdouble 用グループ (MPIDoubleTransferGroup)	4
4.4	InterAll 用グループ (MPIInterAllGroup)	5
5	実装詳細	5
5.1	対応モデル	5
5.2	MPIsingle vs MPIdouble	5
5.3	MPIdouble バッチ処理の詳細	6
5.3.1	origin 計算 (MPIdouble)	6
5.3.2	フェルミオン符号 (MPIdouble)	6
5.3.3	状態の有効性チェック	6
5.3.4	HubbardGC vs Hubbard (canonical)	6
5.4	origin 計算	7
5.4.1	転送項 (Hubbard/SpinlessFermion)	7
5.4.2	InterAll 項	7
5.5	フェルミオン符号	7

6	性能評価	7
6.1	削減効果 (MPIsingle)	7
6.2	MPIdouble テスト結果	7
6.3	デバッグ出力	8
6.4	期待される効果	8
7	使用方法	8
7.1	ビルド	8
7.2	実行	8
8	ソースコード構成	8
9	まとめ	8
9.1	実装済み機能	9
9.2	今後の拡張	9

1 はじめに

HPhi は量子格子モデルの厳密対角化を行うシミュレータである。大規模系の計算では MPI による並列化が必須であり、ハミルトニアン $\hat{H}$ の行列ベクトル積 $\hat{H}|\psi\rangle$ の計算においてプロセス間通信が発生する。

本最適化では、以下の問題に対処する：

- 各転送項が個別に MPI_Sendrecv を呼び出すことによる通信オーバーヘッド
- 同じ通信相手への複数回の重複した通信

2 問題の背景

2.1 HPhi における MPI 並列化

HPhi では、ヒルベルト空間の基底状態を各 MPI プロセスに分散して格納する。 N サイト系で N_{proc} プロセスを使用する場合、各プロセスは $N_{\text{site}} = N - \log_2(N_{\text{proc}})$ 個のサイトを「ローカルサイト」として管理し、残りを「プロセス間サイト」として扱う。

2.2 転送項の MPI 通信

ホッピング項 $c_i^\dagger c_j$ において、サイト i がローカル、サイト j がプロセス間の場合 (MPIsingle)、通信相手 (origin) は以下で計算される：

$$\text{origin} = \text{myrank} \oplus \text{mask} \quad (1)$$

ここで $\oplus$ は XOR 演算、mask は $T_{\text{pow}}[j]$ (サイト j のビットマスク) である。

2.3 従来実装の問題点

従来の実装では、各転送項が個別に MPI_Sendrecv を呼び出していた：

```
1 for (i = 0; i < N_transfers; i++) {  
2     origin = compute_origin(transfer[i]);  
3     MPI_Sendrecv(v1, ..., origin, ...); // 毎回通信  
4     process_transfer(transfer[i], v1buf);  
5 }
```

Listing 1: 従来の MPIsingle 処理 (概念図)

例えば、同じ origin への転送が N 個ある場合、 N 回の MPI_Sendrecv が発生する。これは MPI 通信のレイテンシが支配的な場合に大きなオーバーヘッドとなる。

3 最適化手法：バッチ処理

3.1 基本アイデア

同じ origin を持つ転送項をグループ化し、1 回の MPI_Sendrecv で必要なデータを取得した後、全ての転送項をローカルで処理する。

最適化前	最適化後
転送 1 → MPI 通信	グループ化 → 1 回の MPI 通信 → 転送 1,2,3 をローカル処理
転送 2 → MPI 通信	
転送 3 → MPI 通信	

Figure 1: バッチ処理の概念図 (同じ origin への 3 転送の場合)

3.2 アルゴリズム

Algorithm 1 MPI バッチ処理アルゴリズム

```
1: 初期化フェーズ (1 回のみ実行) :
2: for 各転送項  $t$  do
3:    $\text{origin} \leftarrow \text{myrank} \oplus \text{mask}[t]$ 
4:   転送  $t$  を  $\text{origin}$  別にグループ化
5:   係数・ローカルマスク等を事前計算
6: end for
7:
8: 実行フェーズ (毎イテレーション) :
9: for 各グループ  $g$  do
10:  MPI_Sendrecv( $v1$ ,  $\text{origin}[g]$ ,  $v1\text{buf}$ ) ▷ 1 回の通信
11:   for グループ  $g$  内の各転送  $t$  do
12:      $v1\text{buf}$  を使って転送  $t$  を計算 ▷ ローカル処理
13:   end for
14: end for
```

4 データ構造

4.1 転送項グループ (MPITransferGroup)

```
1 typedef struct {
2     int origin;           // 通信相手のMPIランク
3     int mask;             // ビットマスク
4     int num_transfers;    // このグループの転送数
5     int *transfer_indices; // 転送インデックス配列
6     double complex *coefficients; // 事前計算した係数
7     unsigned long int *local_mask; // ローカルサイトマスク
8     unsigned long int *state1check; // 状態チェック用マスク
9     unsigned long int *bit1diff;   // フェルミオン符号計算用
10 } MPITransferGroup;
```

Listing 2: MPITransferGroup 構造体

4.2 バッチコンテナ (MPIBatchedTransfers)

```
1 typedef struct {
2     int num_groups;       // グループ数 (ユニーク origin 数)
3     MPITransferGroup *groups; // グループ配列
4     int is_initialized;   // 初期化フラグ
5 } MPIBatchedTransfers;
```

Listing 3: MPIBatchedTransfers 構造体

4.3 MPIdouble 用グループ (MPIDoubleTransferGroup)

MPIdouble では両サイトがプロセス間のため、ローカルサイト情報は不要:

```
1 typedef struct {
2     int origin;           // 通信相手のMPIランク
3     int num_transfers;    // このグループの転送数
```

```

4     int *transfer_indices;           // 転送インデックス配列
5     double complex *coefficients;   // 事前計算した係数（符号含む）
6 } MPIDoubleTransferGroup;
7
8 typedef struct {
9     int num_groups;                 // グループ数
10    MPIDoubleTransferGroup *groups;  // グループ配列
11    int is_initialized;              // 初期化フラグ
12 } MPIBatchedDoubleTransfers;

```

Listing 4: MPIDoubleTransferGroup 構造体

4.4 InterAll 用グループ (MPIInterAllGroup)

4 体相互作用 $c_i^\dagger c_j c_k^\dagger c_l$ 用の構造体:

```

1 typedef struct {
2     int origin;                     // 通信相手
3     int num_interall;               // InterAll 項数
4     int *interall_indices;          // インデックス配列
5     double complex *coefficients;   // 係数
6     int *is_hermite;                // エルミート共役フラグ
7     unsigned long int *tmp_isite1;  // 事前計算サイト情報
8     unsigned long int *tmp_isite2;
9     unsigned long int *tmp_isite3;
10    unsigned long int *tmp_isite4;
11    // ... (Tpow 値も格納)
12 } MPIInterAllGroup;

```

Listing 5: MPIInterAllGroup 構造体

5 実装詳細

5.1 対応モデル

モデル	MPIsingle バッチ	MPIdouble バッチ	InterAll バッチ
SpinlessFermionGC	✓	—	—
SpinlessFermion (canonical)	✓	—	—
HubbardGC	✓	✓	✓
Hubbard (canonical)	✓	✓	—

Table 1: バッチ処理対応状況

5.2 MPIsingle vs MPIdouble

- **MPIsingle**: 一方のサイトがローカル、他方がプロセス間
- **MPIdouble**: 両方のサイトがプロセス間

MPIdouble は MPIsingle よりも単純で、受信後の処理が単なるベクトルスケールリングになる:

$$v_0[j] += \text{trans} \times v_{\text{buf}}[j] \quad (2)$$

5.3 MPIdouble バッチ処理の詳細

5.3.1 origin 計算 (MPIdouble)

両サイトがプロセス間の場合、origin は以下で計算される：

$$\text{origin} = \text{myrank} \oplus (\text{mask}_1 + \text{mask}_2) \quad (3)$$

ここで $\text{mask}_1 = T_{\text{pow}}[2 \cdot \text{site}_1 + \sigma_1]$ 、 $\text{mask}_2 = T_{\text{pow}}[2 \cdot \text{site}_2 + \sigma_2]$ である。

5.3.2 フェルミオン符号 (MPIdouble)

MPIdouble でのフェルミオン符号は以下で計算される：

```

1 if (mask2 > mask1) bitdiff = mask2 - mask1 * 2;
2 else bitdiff = mask1 - mask2 * 2;
3 SgnBit((unsigned long int)(origin & bitdiff), &Fsgn);

```

Listing 6: MPIdouble フェルミオン符号の計算

5.3.3 状態の有効性チェック

各プロセスで転送が有効かどうかは、origin の状態で判定される：

- $\text{state}_1 = 0$ かつ $\text{state}_2 = \text{mask}_2$: 順方向 ($c_1^\dagger c_2$)、係数 $= -F_{\text{sgn}} \times t$
- $\text{state}_1 = \text{mask}_1$ かつ $\text{state}_2 = 0$: 逆方向 ($c_2^\dagger c_1$)、係数 $= -F_{\text{sgn}} \times t^*$
- その他: 無効状態、係数 $= 0$

重要: 全プロセスが同じ通信パターンを持つ必要があるため、無効状態でも MPI 通信は実行し、係数を 0 にすることで寄与をなくす。

5.3.4 HubbardGC vs Hubbard (canonical)

項目	HubbardGC	Hubbard (canonical)
list_1 交換	不要	必要
インデックス計算	直接 j	GetOffComp(list_1buf[j])
処理の複雑さ	単純	やや複雑

Table 2: HubbardGC と Hubbard (canonical) の MPIdouble 処理の違い

Hubbard (canonical) では、粒子数保存のため状態空間が制限される。そのため、list_1 (状態インデックス配列) を交換し、GetOffComp 関数でターゲットインデックスを計算する必要がある：

```

1 // list_1の交換
2 MPI_Sendrecv(list_1, ..., list_1buf, ...);
3 MPI_Sendrecv(tmp_v1, ..., v1buf, ...);
4
5 // 各要素の処理
6 for (j = 1; j <= idim_max_buf; j++) {
7     GetOffComp(list_2_1, list_2_2, list_1buf[j],
8               irght, ilft, ihfbit, &ioff);
9     tmp_v0[ioff] += trans * v1buf[j];
10 }

```

Listing 7: Hubbard canonical MPIdouble の処理

5.4 origin 計算

5.4.1 転送項 (Hubbard/SpinlessFermion)

サイト i (ローカル) からサイト j (プロセス間) への転送:

$$\text{origin} = \text{myrank} \oplus T_{\text{pow}}[2j + \sigma] \quad (4)$$

ここで σ はスピン (Hubbard モデル)、SpinlessFermion では $\sigma = 0$ 。

5.4.2 InterAll 項

4 サイト相互作用では、CheckBit_InterAllPE 関数によりビット操作を適用して origin を計算:

$$\text{origin} = A_1^\dagger C_2 A_3^\dagger C_4 |\text{myrank}\rangle \quad (5)$$

5.5 フェルミオン符号

フェルミオン系では、演算子の交換により符号が変化する。バッチ処理では初期化時に符号を事前計算:

```
1 // プロセス間部分の符号
2 int bit2diff = mask2 - 1;
3 int Fsgn;
4 SgnBit((unsigned long int)(origin & bit2diff), &Fsgn);
5
6 // 係数に符号を含める
7 coefficients[t] = -(double)Fsgn * trans_coeff;
```

Listing 8: フェルミオン符号の事前計算

6 性能評価

6.1 削減効果 (MPIsingle)

4×2 格子の HubbardGC モデル (8 サイト、4 MPI プロセス) での測定結果:

項目	最適化前	最適化後
MPIsingle 転送数	6	6
MPI_Sendrecv 回数	6	2
削減率	—	3.0 倍

Table 3: MPI 通信回数の削減 (MPIsingle)

6.2 MPIdouble テスト結果

4 サイト鎖の Hubbard モデル (16 MPI プロセス) での MPIdouble バッチ処理テスト:

モデル	MPIdouble 転送数	グループ数	エネルギー
HubbardGC	2	2	-3.41855071887385
Hubbard (canonical)	2	2	-2.10274848346207

Table 4: MPIdouble バッチ処理のテスト結果

16 MPI プロセスを使用することで、サイト 2,3 がプロセス間サイトとなり、これらのサイト間の転送が MPIdouble として処理される。

6.3 デバッグ出力

実行時に以下のようなバッチ処理統計が出力される：

```
[MPI Batching] HubbardGC MPIdouble: 2 transfers -> 2 groups (1.0x reduction)
[MPI Batching] HubbardGC: 4 MPIsingle transfers -> 4 groups (1.0x reduction)
[MPI Batching] HubbardGC InterAll: No off-diagonal terms
```

Hubbard (canonical) の場合：

```
[MPI Batching] Hubbard MPIdouble: 2 transfers -> 2 groups (1.0x reduction)
[MPI Batching] HubbardGC: 4 MPIsingle transfers -> 4 groups (1.0x reduction)
```

6.4 期待される効果

- 最良ケース: 同じ origin への N 転送 $\rightarrow$ MPI 呼び出し N 回が 1 回に
- 典型的改善: 2D/3D モデルで顕著（多くの転送が同じ origin を共有）
- オーバーヘッド: 初期化時に 1 回のグループ化コスト（無視できる程度）

7 使用方法

バッチ処理は自動的に有効化される。特別な設定は不要である。

7.1 ビルド

```
mkdir build && cd build
cmake .. -DUSE_MPI=ON
make
```

7.2 実行

通常通り MPI で実行：

```
mpirun -np 4 ./HPhi -s stan.in
```

8 ソースコード構成

ファイル	内容
src/include/mltplyMPIBatched.h	データ構造・関数宣言
src/mltplyMPIBatched.c	バッチ処理実装
src/mltplyHubbard.c	Hubbard モデル（バッチ呼び出し）
src/mltplySpinless.c	SpinlessFermion モデル（バッチ呼び出し）

Table 5: 関連ソースファイル

9 まとめ

本最適化により、HPhi の MPI 通信効率を改善した。同じ通信相手を持つ複数の転送項や InterAll 項をグループ化し、MPI_Sendrecv 呼び出し回数を削減することで、特に通信レイテンシが支配的な環境での性能向上が期待できる。

9.1 実装済み機能

- MPIsingle バッチ処理: SpinlessFermionGC, SpinlessFermion, HubbardGC, Hubbard
- MPIdouble バッチ処理: HubbardGC, Hubbard (canonical)
- InterAll バッチ処理: HubbardGC

9.2 今後の拡張

今後の拡張として以下が考えられる：

- Hubbard (canonical) の InterAll バッチ処理
- Spin モデルへの適用
- SpinlessFermion の MPIdouble バッチ処理

謝辞

本実装は Claude Opus 4.5 との協働により開発された。